

Phase 1: Foundations of the Array ADT

1. Big Idea of Phase 1

Phase 1 teaches the basic foundation needed before writing array operations like append, insert, delete, search, and reverse.

The main goal is to understand this idea:

Array ADT = array storage + size + length + operations

A normal array only gives memory for storing values. An Array ADT adds extra information and functions so we can manage the array properly.

2. What is an ADT?

ADT means **Abstract Data Type**.

In simple words:

ADT = Data + Operations on that data

For example, a normal C array gives data storage:

```
int A[10];
```

This creates space for 10 integers.

But it does not automatically provide useful operations like:

```
display()  
append()  
insert()  
delete()  
search()  
get()  
set()  
max()
```

```
min()  
reverse()
```

When we combine the array storage with custom functions, we get an **Array ADT**.

So, an Array ADT is not only the array. It is the array plus the rules and operations we write for it.

3. Why a Plain Array is Not a Complete ADT

A plain array only stores elements.

Example:

```
int A[10];
```

This gives 10 spaces in memory, but it does not know:

- how many elements are actually being used
- where the next new element should go
- how to insert a value safely
- how to delete a value safely
- how to display only valid elements
- how to search using a custom function

That is why we add extra information like `size` and `length`, and we write functions for operations.

4. The Three Core Parts of an Array ADT

Every Array ADT needs three main parts:

```
A        // actual array data  
size     // total capacity of the array  
length   // number of valid elements currently used
```

4.1 `A`: The Actual Array

`A` stores the real values.

Example:

```
int A[10] = {2, 4, 6, 8};
```

The array has space for 10 values, but only 4 values are currently meaningful.

4.2 `size` : Maximum Capacity

`size` means the maximum number of values the array can hold.

Example:

```
size = 10
```

This means the array has 10 available slots.

4.3 `length` : Current Number of Valid Elements

`length` means how many values are actually being used right now.

Example:

```
length = 4
```

This means only 4 values are valid data.

5. Difference Between `size` and `length`

This is one of the most important ideas in Phase 1.

```
size  = total capacity  
length = currently used elements
```

Example:

```
int A[10] = {2, 4, 6, 8};
```

Memory view:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	2	4	6	8	_	_	_	_	_	_

Here:

```
size = 10
length = 4
```

The empty slots exist in memory, but they are not valid logical elements yet.

Simple comparison

Think of an array like a notebook.

- `size` is the total number of pages.
- `length` is the number of pages already written.

A notebook can have 100 pages but only 5 written pages.

In the same way, an array can have size 10 but length 4.

6. Why an Array Can Have `size = 10` but `length = 0`

Suppose we create this:

```
int A[10];
```

This reserves space for 10 integers.

But no meaningful values have been inserted yet.

So:

```
size = 10
length = 0
```

The array has capacity, but it is logically empty.

This is important because unused memory slots should not be treated as real data.

7. Static Array Creation

A static array is created with a fixed size.

Example:

```
int A[10];
```

This creates an array of 10 integers.

Key points about static arrays

- Created on the stack.
- Size is decided at compile time.
- Size is fixed.
- Simple to use.
- Less flexible because the size cannot easily change while the program runs.

Example:

```
int marks[5];
```

This array can store 5 integers. The size is already fixed as 5.

8. Dynamic Array Creation

A dynamic array is created while the program is running.

This is useful when the user decides the array size.

In C++

```
int *A;  
int size = 10;  
A = new int[size];
```

In C

```
int *A;  
int size = 10;  
A = (int *)malloc(size * sizeof(int));
```

Key points about dynamic arrays

- Created on the heap.
- Size can be decided at run time.
- More flexible than static arrays.
- Requires memory allocation.
- In C, `malloc()` is used.
- In C++, `new` can be used.

9. Stack vs Heap

Stack

The stack is used for simple local variables and fixed-size arrays.

Example:

```
int A[10];
```

This is usually stored on the stack.

Heap

The heap is used for memory created dynamically while the program is running.

Example:

```
A = (int *)malloc(size * sizeof(int));
```

This creates memory in the heap.

Simple difference

Feature	Stack	Heap
Used for	Fixed/local data	Dynamic memory

Feature	Stack	Heap
Size decided	Usually compile time	Run time
Example	<code>int A[10];</code>	<code>malloc()</code>
Flexibility	Less flexible	More flexible

10. Pointer Used for Dynamic Array

For a dynamic array, we use a pointer.

```
int *A;
```

This does not create the array yet.

It only creates a pointer variable that can store the address of an integer.

Then we allocate memory:

```
A = (int *)malloc(size * sizeof(int));
```

Now `A` points to the first element of the heap array.

Important sequence

```
int *A;
```

This creates only the pointer.

```
A = (int *)malloc(size * sizeof(int));
```

This creates the actual memory block and connects `A` to it.

11. First Array ADT Structure in C

In C, we can group the array information using a `struct`.

```
struct Array {  
    int *A;  
    int size;  
    int length;  
};
```

Meaning of each member

```
int *A;
```

`A` points to the first element of the dynamic array.

```
int size;
```

`size` stores the maximum capacity.

```
int length;
```

`length` stores the current number of valid elements.

12. Creating an Array ADT Dynamically

Basic program:

```
#include <stdio.h>  
#include <stdlib.h>  
  
struct Array {  
    int *A;  
    int size;  
    int length;  
};  
  
int main() {  
    struct Array arr;  
  
    printf("Enter size of array: ");  
    scanf("%d", &arr.size);  
  
    arr.A = (int *)malloc(arr.size * sizeof(int));
```



```
    arr.length = 0;

    return 0;
}
```

Explanation

```
struct Array arr;
```

This creates one Array ADT variable named `arr`.

```
scanf("%d", &arr.size);
```

This asks the user for the maximum array size.

```
arr.A = (int *)malloc(arr.size * sizeof(int));
```

This creates the array in heap memory.

```
arr.length = 0;
```

This says the array currently has no valid elements.

13. Filling the Array with Values

After creating the array, we can ask the user how many values they want to enter.

```
int n;

printf("How many numbers do you want to enter? ");
scanf("%d", &n);

printf("Enter all elements:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &arr.A[i]);
}

arr.length = n;
```

Example input

```
Enter size of array: 10
How many numbers do you want to enter? 5
Enter all elements:
2 3 4 5 6
```

Now the array has:

```
size = 10
length = 5
```

Memory view:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	2	3	4	5	6	_	_	_	_	_

Only indexes `0` to `4` contain valid logical data.

14. Why We Must Not Print Up to `size`

If we print from index `0` to `size - 1`, we may print unused or garbage values.

Wrong idea:

```
for (int i = 0; i < arr.size; i++) {
    printf("%d ", arr.A[i]);
}
```

This prints all capacity slots, including unused ones.

Correct idea:

```
for (int i = 0; i < arr.length; i++) {
    printf("%d ", arr.A[i]);
}
```

This prints only valid elements.

15. Display Function

The display function prints the valid elements of the array.

```
void display(struct Array arr) {  
    printf("Elements are:\n");  
  
    for (int i = 0; i < arr.length; i++) {  
        printf("%d ", arr.A[i]);  
    }  
  
    printf("\n");  
}
```

Important detail

The loop uses:

```
i < arr.length
```

It does not use:

```
i < arr.size
```

Because `length` tells us how many real values exist.

16. Why `display()` Receives the Structure by Value

The display function only reads and prints the array.

It does not change:

- array values
- size
- length

So it can receive the structure by value:

```
void display(struct Array arr)
```

Later operations like append, insert, and delete will need pointers because they modify the array.

17. Full Phase 1 Beginner Program

```
#include <stdio.h>
#include <stdlib.h>

struct Array {
    int *A;
    int size;
    int length;
};

void display(struct Array arr) {
    printf("Elements are:\n");

    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }

    printf("\n");
}

int main() {
    struct Array arr;
    int n;

    printf("Enter size of array: ");
    scanf("%d", &arr.size);

    arr.A = (int *)malloc(arr.size * sizeof(int));
    arr.length = 0;

    printf("How many numbers do you want to enter? ");
    scanf("%d", &n);

    if (n > arr.size) {
        printf("Too many elements. Cannot exceed array size.\n");
        return 1;
    }

    printf("Enter all elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr.A[i]);
    }

    arr.length = n;
}
```

```
    display(arr);

    free(arr.A);

    return 0;
}
```

What this program does

1. Defines the Array ADT structure.
2. Asks the user for maximum array size.
3. Allocates heap memory using `malloc()`.
4. Sets `length` to 0.
5. Reads `n` elements from the user.
6. Updates `length` to `n`.
7. Displays only valid elements.
8. Frees the heap memory using `free()`.

18. Why We Use `free(arr.A)`

When memory is created using `malloc()`, it should be released after use.

```
free(arr.A);
```

This gives the heap memory back to the system.

In small beginner programs, forgetting this may not seem serious, but in larger programs it can cause memory leaks.

19. Fixed-Size Array Version

Later, the curriculum switches to a fixed-size version for easier tracing.

Dynamic version:

```
struct Array {
    int *A;
    int size;
```

```
    int length;  
};
```

Fixed-size version:

```
struct Array {  
    int A[20];  
    int size;  
    int length;  
};
```

Here, the array is directly inside the structure.

Example:

```
struct Array arr = {{2, 3, 4, 5, 6}, 20, 5};
```

This means:

```
A      = [2, 3, 4, 5, 6, ...]  
size   = 20  
length = 5
```

20. Why the Fixed-Size Version is Easier for Debugging

With the dynamic version:

```
int *A;
```

`A` is only a pointer to heap memory.

Some debuggers may show only the first value or may not clearly show the whole heap array.

With the fixed-size version:

```
int A[20];
```

The array is directly inside the structure, so it is easier to inspect during debugging.

That is why beginners often use the fixed-size structure while learning operations.

21. Memory Trace for Static Array

Code:

```
int staticArray[10];
```

What happens:

- 10 integer slots are reserved immediately.
 - The memory is usually on the stack.
 - The size is fixed.
 - If no values are inserted, logical length can still be 0.
-

22. Memory Trace for Dynamic Array

Code:

```
int *A;  
int size = 10;  
A = (int *)malloc(size * sizeof(int));
```

Step-by-step:

1. `int *A;` creates a pointer.
2. No array exists yet.
3. `malloc()` creates heap memory for 10 integers.
4. `A` stores the address of the first slot.
5. `length` should be set to 0 at the beginning.

Important:

```
Pointer created does not mean array created.
```

The array is created only after memory allocation.

23. Operations Introduced in Phase 1

Phase 1 introduces the idea that an Array ADT will have many operations.

Output and update operations

```
Display
Append / Add
Insert
Delete
```

Search and access operations

```
Search
Get
Set
```

Computation operations

```
Max
Min
```

Rearrangement operations

```
Reverse
Shift
Rotate
```

In Phase 1, we mainly implement display and prepare the structure for the other operations.

24. Complexity in Phase 1

24.1 Array Creation

```
Time complexity:  $O(1)$ 
Space complexity:  $O(N)$ 
```

The setup work is constant, but the memory reserved depends on the requested size N .

24.2 Filling the Array

Time complexity: $O(N)$
Space complexity: $O(1)$ auxiliary space

Why time is $O(N)$:

The program uses a loop to read N elements.

Why extra space is $O(1)$:

Only a few extra variables like i and n are used.

24.3 Displaying the Array

Time complexity: $O(N)$

Why:

Display visits each valid element once.

25. Common Beginner Mistakes in Phase 1

Mistake 1: Confusing `size` and `length`

Wrong thinking:

If size is 10, the array has 10 valid elements.

Correct thinking:

size is capacity, length is valid data count.

Mistake 2: Printing up to `size`

Wrong:

```
for (int i = 0; i < arr.size; i++)
```

Correct:

```
for (int i = 0; i < arr.length; i++)
```

Mistake 3: Forgetting to Set `length`

After creating the array:

```
arr.length = 0;
```

After reading `n` elements:

```
arr.length = n;
```

Mistake 4: Thinking a Pointer Automatically Creates an Array

This only creates a pointer:

```
int *A;
```

The actual array is created here:

```
A = (int *)malloc(size * sizeof(int));
```

Mistake 5: Forgetting to Free Heap Memory

If memory is allocated using `malloc()`, release it later:

```
free(arr.A);
```

26. Phase 1 Summary

Phase 1 teaches that an Array ADT is more than a normal array.

A normal array gives storage, but an Array ADT adds structure and operations.

The most important parts are:

```
A      = actual array storage
size   = maximum capacity
length = current number of valid elements
```

The most important rule is:

```
Use length when working with valid elements.
Use size only when checking total capacity.
```

The first useful function is `display()`, and it must loop from index `0` to `length - 1`.

27. Phase 1 Checkpoint Questions

You are ready for Phase 2 when you can answer these:

1. What does ADT mean?
 2. Why is an array alone not a complete ADT?
 3. What are the three main parts of an Array ADT?
 4. What is the difference between `size` and `length`?
 5. Why can an array have `size = 10` and `length = 0`?
 6. What is the difference between static and dynamic array creation?
 7. Where is a static array usually stored?
 8. Where is a dynamic array stored?
 9. What does `int *A` mean?
 10. What does `malloc()` do?
 11. Why should `display` use `length`, not `size`?
 12. Why is the fixed-size struct easier to debug?
 13. What is the time complexity of filling `N` elements?
 14. What is the time complexity of displaying `N` elements?
 15. Why should heap memory be freed?
-

28. One-Line Final Memory

```
Phase 1 is about understanding how an Array ADT stores data, tracks capacity
with size, tracks valid elements with length, and prepares the structure for
future operations.
```